

3. PROJECT DESIGN PHASE

ComicCraft System Architecture & Structure Specification

Document: Technical System Architecture Design

Project: ComicCraft AI

1. Introduction

Project design defines the technical architecture and modular structure of the platform. ComicCraft is systematically organized around three primary layers: an interactive web-based frontend, a robust FastAPI backend service, and specialized AI integration modules handling textual narrative and visual generation.

2. Objectives

- Design clean and intuitive HTML/CSS templates for web interactions.
- Design RESTful FastAPI route endpoints for web rendering and payload processing.
- Decouple AI responsibilities into dedicated Python integration modules.
- Architect seamless end-to-end data pipelines for comic panel generation.
- Design an automated, server-side PDF layout compilation and export pipeline.
- Establish a maintainable, enterprise-ready repository directory layout.

3. Phase Workflow

Frontend UI → FastAPI Backend → Gemini Outline → Gemini Story → Stable Diffusion Images → Layout Builder → PDF Exporter

4. Detailed Explanation

The FastAPI service coordinates routing and server-side template rendering using Jinja2 templates. To ensure maintainability and separation of concerns, story outline creation, script generation, image diffusion, and PDF compilation are segmented into single-responsibility Python modules.

```
from fastapi import FastAPI
from fastapi.templating import Jinja2Templates

app = FastAPI(title="ComicCraft")
templates = Jinja2Templates(directory="templates")
```

5. GitHub-Ready Code / Directory Structure

```
ComicCraft/
├── app/
│   ├── main.py
│   ├── routes.py
│   ├── gemini_flash.py
│   ├── gemini_pro.py
│   ├── image_generator.py
│   ├── layout_builder.py
│   └── exporters.py
├── templates/
│   ├── index.html
│   ├── comic_preview.html
│   └── export_success.html
├── static/
│   ├── panels/
│   └── exports/
├── requirements.txt
├── .env.example
└── README.md
```

6. GitHub Files / Deliverables

Directory / File Path	Purpose / Deliverable Description
docs/architecture.md	Architectural documentation covering core layers and software workflows.
docs/data-flow.md	Detailed data processing pipeline and model interaction schemas.
app/	Core Python codebase (FastAPI routes, AI generators, PDF exporter).
templates/	Jinja2 HTML templates for dynamic user interface pages.
static/	Static assets, generated image panels, and exportable PDF files.

7. Expected Outcome

At the end of Phase 3, the corresponding part of ComicCraft should be clearly documented, implemented, or verified according to the purpose of the phase. The actual completion status should be based on the student's repository and execution setup.

8. Phase Conclusion

Phase 3 provides a structured foundation for the next stage of ComicCraft development, keeping the project organized, scalable, and prepared for GitHub-based collaboration and academic evaluation.